

Database

1. Draw the schema diagram from the given relations.

customer (customer_name, customer_street, customer_city)

loan (loan_number, branch_name, amount)

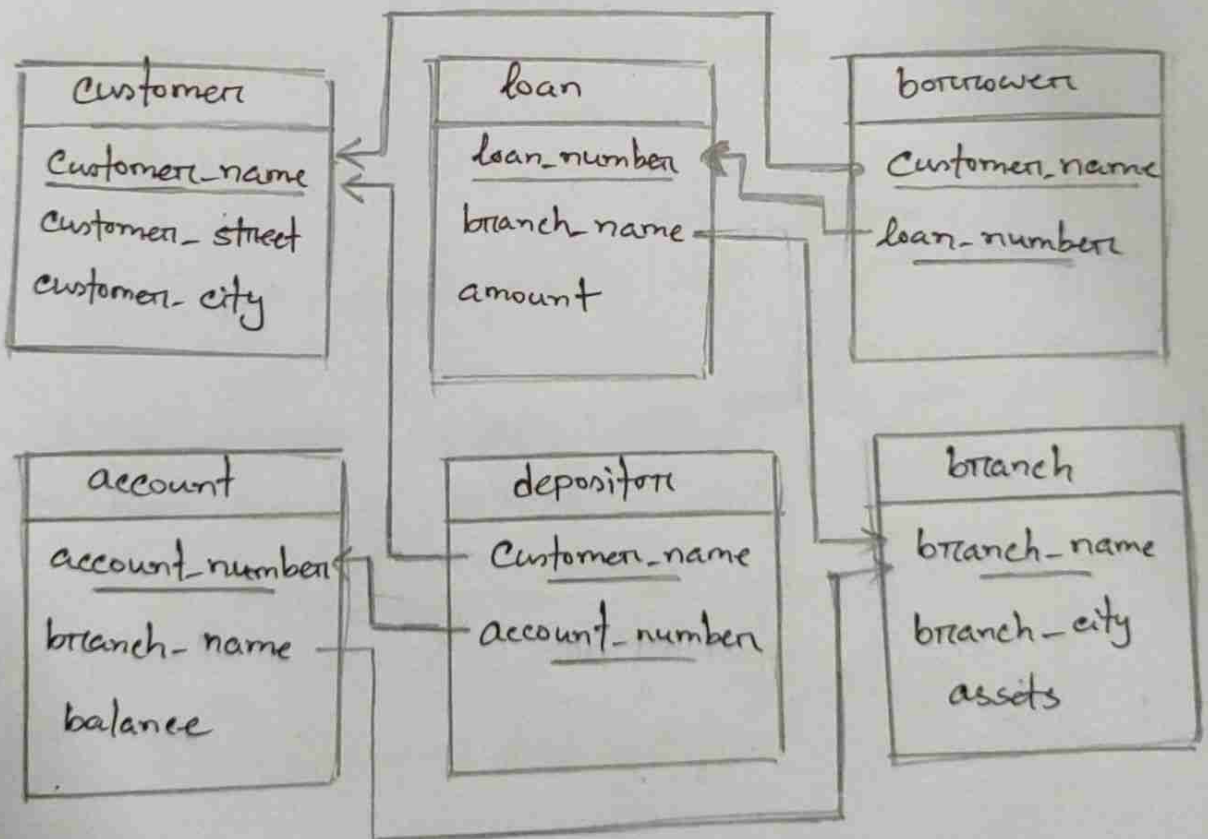
borrower (customer_name, loan_number)

account (account_number, branch_name, balance)

depositor (customer_name, account_number)

branch (branch_name, branch_city, assets)

Schema diagram:



Question 2: Write down the SQL and relational Algebra expressions for the questions below.

1. Find the average loan amount from each branch.

SQL: `SELECT branch_name, AVG(amount) FROM loan
WHERE GROUP BY branch_name;`

R.A: $\sigma_{\text{branch_name } G_1 \text{ AVG}(\text{amount})} (\text{loan})$

2. Write down the SQL and to show the details of a customer whose street name has two consecutive 'S'.

SQL: `SELECT * FROM customer WHERE customer_street
LIKE "%SS%";`

RA: $\sigma_{\text{customer_street} = "%SS%"} (\text{customer})$

3. Find all customers who have a loan from the bank, find their names and loan numbers.

SQL: `SELECT customer_name, loan_number from borrower.`

R.A: $\pi_{\text{customer_name}, \text{loan_number}} (\text{Borrower})$

4. Find the list of all customers in alphabetic order who have a loan in the 'Perryridge' branch.

SQL: $\text{SELECT } \overset{\text{DISTINCT}}{\wedge} \text{customer_name FROM loan NATURAL JOIN borrower}$
 $\text{WHERE branch_name = "Perryridge" ORDER BY customer_name ASC};$

R.A:

5. Find all customers having a loan, an account, or both at the bank.

SQL: $\text{SELECT customer_name FROM borrower UNION SELECT customer_name FROM depositor}$

R.A: $\pi_{\text{customer_name}} (\text{Borrower}) \cup \pi_{\text{customer_name}} (\text{Depositor})$

6. Find the names of all customers whose street address includes the substring "Main":

SQL: $\text{SELECT customer_name FROM customer WHERE customer_street LIKE "\%.Main\%";}$

R.A: $\pi_{\text{customer_name}} \left(\sigma_{\text{customer_street} = "\%.Main\%"} (\text{Customer}) \right)$

Q 7. Find the average loan amount from each branch where the average loan amount is greater than 1500.

SQL: SELECT branch_name, AVG(amount) "Avg amount"
from loan GROUP BY branch_name HAVING AVG(amount) > 1500;

R.A:

Q. Count the number of tuples in customer relations.

SQL: SELECT Count(customer_name) FROM customer
(Customer)

R.A: G Count(customer_name)

9. Find the average account balance, maximum account balance at each branch.

SQL: SELECT branch_name, AVG(balance), MAX(balance)
FROM account GROUP BY branch_name;

R.A: branch_name G1 AVG(balance). MAX(balance) (Account)

10. Find the names of all those customers who have a loan at Perryridge branch.

SQL: SELECT customer-names FROM loan NATURAL JOIN borrower WHERE branch-name = "Perryridge";

R.A: $\pi_{\text{customer-name}} (\sigma_{\text{branch-name} = \text{"Perryridge"}} (\text{Loan} \bowtie \text{Borrower}))$

11. Delete the records of all accounts with balances below the average of that bank.

SQL: DELETE FROM account WHERE balance < (select AVG(balance) FROM account)

R.A: $\text{Account} \leftarrow \text{Account} - \sigma_{\text{balance} < \text{AVG}(\text{balance})} (\text{Account})$

12. Consider the following tables "loan" and "borrowers".

Perform RIGHT OUTER JOIN, LEFT OUTER JOIN, INNER JOIN and NATURAL JOIN operations and show the results.

Loan		
loan-number	branch-name	amount
L-11	Round Hill	900
L-14	Downtown	1500
L-15	Perryridge	1500
L-16	Perryridge	1300
L-17	Downtown	1000
L-23	Redwood	2000
L-93	Mianus	500

borrowers	
customer-name	loan-number
Adams	L-16
Curry	L-93
Hayes	L-15
Johnson	L-14
Jones	L-17
Smith	L-11
Smith	L-23
Williams	L-17

Right Outer Join (R)

SELECT * FROM loan NATURAL RIGHT OUTER JOIN borrowers.

loan-number	customer-name	branch-name	amount
L-16	Adams	Perryridge	1300
L-93	Curry	Mianus	500
L-15	Hayes	Perryridge	1500
L-14	Johnson	Downtown	1500
L-17	Jones	Downtown	1000
L-11	Smith	Round Hill	900
L-23	Smith	Redwood	2000
L-17	Williams	Downtown	1000

LEFT OUTER JOIN (⇐)

SQL: SELECT * FROM loan NATURAL LEFT OUTER JOIN borrower;

Loan-number	Branch-name	amount	Customer-name
L-11	Round Hill	900	Smith
L-14	Downtown	1500	Johnson
L-15	Perryridge	1500	Hays
L-16	Perryridge	1300	Adams
L-17	Downtown	1000	Jones
L-23	Redwood	2000	Smith
L-93	Mianus	500	Curry

INNER JOIN (⇔)

SQL: SELECT * FROM loan INNER JOIN borrower;

loan-number	branch-name	amount	customer-name	loan-number
L-11	Round Hill	900	Adams	L-16
L-14	Downtown	1500	Adams	L-16
L-15	Perryridge	1500	Adams	L-16
L-16	Perryridge	1300	Adams	L-16
L-17	Downtown	1000	Adams	L-16
L-23	Redwood	2000	Adams	L-16
L-93	Mianus	500	Adams	L-16
L-11	Round Hill	900	Curry	L-93
L-14	Downtown	1500	Curry	L-93
L-15	Perryridge	1500	Curry	L-93
L-16	Perryridge	1300	Curry	L-93
L-17	Downtown	1000	Curry	L-93
L-23	Redwood	2000	Curry	L-93
L-93	Mianus	500	Curry	L-93

PTD

loan number	branch name	amount	customer name	loan number
L-11	Round Hill	900	Hayes	L-15
L-14	Downtown	1500	Hayes	L-15
L-15	Perryridge	1500	Hayes	L-15
L-16	Perryridge	1300	Hayes	L-15
L-17	Downtown	1000	Hayes	L-15
L-23	Redwood	2000	Hayes	L-15
L-93	Mianus	500	Hayes	L-15
L-11	Round Hill	900	Johnson	L-14
L-14	Downtown	1500	Johnson	L-14
L-15	Perryridge	1500	Johnson	L-14
L-16	Perryridge	1300	Johnson	L-14
L-17	Downtown	1000	Johnson	L-14
L-23	Redwood	2000	Johnson	L-14
L-93	Mianus	500	Johnson	L-14

Natural Join (X)

SQL: SELECT * FROM loan NATURAL JOIN borrowers;

loan number	branch name	amount	customer name
L-16	Perryridge	1300	Adams
L-93	Mianus	500	Curry
L-15	Perryridge	1500	Hayes
L-14	Downtown	1500	Johnson
L-17	Downtown	1000	Jones
L-11	Round Hill	900	Smith
L-23	Redwood	2000	Smith
L-17	Downtown	1000	Williams

Qus 3: Demonstrate the output of the above questions
from Qus 2 (Q1-Q-10)

1. Find the average loan amount from each branch

```
SELECT branch_name, AVG(amount) "Average loan"  
FROM loan GROUP BY branch_name;
```

branch_name	Average loan
Downtown	12500.00
Midtown	500.00
Perryridge	1400.00
Redwood	2000.00
Round Hill	900.00

2. Write a query to show the details of a customer whose
street name has two consecutive 'S'.

```
SELECT * FROM Customer WHERE customer_street LIKE "%SS%";
```

Customer_name	customer_street	customer_city
Williams	Nassau	Princeton

3. Find all customers who have a loan from the bank, find their names and loan numbers.

SELECT customer_name, loan_number FROM borrower;

customer_name	loan_number
Adams	L-16
Curry	L-93
Hayes	L-15
Johnson	L-14
Jones	L-17
Smith	L-11
Smith	L-23
William	L-17

4. Find the list of all customers in alphabetic order who have a loan in the 'Perryridge' branch.

SELECT DISTINCT customer_name FROM borrower
NATURAL JOIN loan WHERE branch_name = "Perryridge"
ORDER BY customer_name ASC;

customer_name
Adams
Hayes

5. Find all customers having a loan, an account, or both at the bank.

```
SELECT customer_name FROM borrowers UNION  
SELECT customer_name FROM depositors
```

customer_name
Adams
Curry
Hayes
Johnson
Jones
Smith
William
Lindsay
Turner

6. Find the names of all customers whose street address includes the substring 'Main'

```
SELECT customer_name FROM customer WHERE  
customer_street LIKE "%.Main%";
```

customer_name
Hayes
Jones

7. Find the average loan amount from each branch where the average loan amount is greater than 1500

```
SELECT branch_name, AVG(amount) FROM loan
GROUP BY branch_name HAVING AVG(amount) > 1500
```

branch_name	AVG(amount)
Redwood	2000.000

8. Count the number of tuples in customer relation.

```
SELECT COUNT (customer_name) "Num of Tuples" FROM customer
```

Num of Tuples
12

9. Find the average account balance, maximum account balance at each branch.

```
SELECT branch_name, AVG(balance), MAX(balance)
FROM account GROUP BY branch_name
```

branch_name	avg_balance	max_balance
Brighton	825.00	900
Downtown	500.00	500
Mianus	700.00	700
Perryridge	400.00	400
Redwood	700.00	700
Round Hill	350.00	350

10. Find the names of all those customers who have a loan at Pennyridge branch.

```
SELECT customer_name FROM borrowers NATURAL JOIN  
loan WHERE branch_name = "Pennyridge";
```

customer_name
Adams
Hayes

Ques 4: Identify all possible Superkeys and candidate Keys.

Employee (EmpID, Name, Email, Phone, Department)

Superkey: A superkey is the basic key in a database table.

- Any superset of a candidate key is also a superkey.

Ex. Examples: {EmpID}, {Email}, {Phone}, {EmpID, Name},
{EmpID, Email}, {EmpID, Phone}, {Phone, Name} etc.

Candidate Key: The minimal set of one or more attributes which can uniquely identify a tuple/row is known as a candidate key.

Example: {EmpID}, {Email}, {Phone}